Contents

MarkdownPrint -- User Manual

MarkdownPrint is a Swift library that converts Markdown documents into professional PDF files with native Apple typography (San Francisco + Menlo), four visual themes (light, dark, mono, high contrast), navigable outline, clickable links, images, and syntax highlighting. It does not depend on AppKit or UIKit: it works directly on CoreGraphics, CoreText, and PDFKit. Compatible with macOS 13+, iOS 16+, and visionOS 1+.

---

1. Installation

Add the dependency to your `Package.swift`:

```
// swift-tools-version:5.9
import PackageDescription

let package = Package(
name: "MyApp",
platforms: [
.macOS(.v13),
.iOS(.v16)
],
dependencies: [
.package(url: "https://github.com/your-org/MarkdownPrint", from: "1.0.0")
],
targets: [
.target(
name: "MyApp",
dependencies: ["MarkdownPrint"]
)
]
)
```

The library consists of three internal modules:

- `MarkdownPrint` -- full PDF rendering (Apple only)
- `MarkdownPrintCore` -- parsing and layout (cross-platform, usable

on Linux)

- `CMarkdownPrintCore` -- portable C++ core (not imported directly)

---

2. Basic Usage

2.1 The unified API (recommended)

```
import MarkdownPrint

let engine = MarkdownPrintEngine()
let markdown = """
# Hello World

This is a **document** with [links](https://example.com).
"""

let result = try engine.render(markdown)
try result.pdfData.write(to: URL(fileURLWithPath: "output.pdf"))

print(result.diagnostics)
// pages: 1, size: 22 KB, duration: 9 ms
```

`render()` returns a `PDFRenderResult` with `.pdfData`, `.pageCount`, `.diagnostics`, and more.

2.2 With options

```
let result = try engine.render(markdown, options: .init(
pageSize: .a4,
metadata: PDFMetadata(title: "Annual Report", author: "Engineering"),
theme: .dark,
withTOC: true,
fontFamily: .web
))
```

All options have sensible defaults. `RenderOptions.default` gives you A4, light theme, no TOC, Apple fonts.

2.3 Async rendering

```
let result = try await engine.render(
markdown,
options: .init(theme: .dark)
)
```

2.4 With progress reporting

```
let progress = Progress()
let result = try engine.render(
markdown,
options: .default,
progress: progress
)
// progress.fractionCompleted updates as pages render
```

2.5 Parse-only (no rendering)

```
let tokens = engine.tokenize(markdown) // [MarkdownToken]
let blocks = engine.parse(markdown) // [MarkdownBlock]
let layout = engine.layout(markdown, pageSize: .a4)
```

Useful for inspecting document structure or building custom pipelines.

---

3. API Reference

3.1 `RenderOptions`

Central configuration struct. All properties have defaults.

- `pageSize`: `MarkdownPageSize`, default `.a4`. A4, US Letter, or custom.
- `metadata`: `PDFMetadata`, default `.init()`. Title, author, subject, and keywords.
- `baseURL`: `URL?`, default `nil`. Base path for relative images.
- `theme`: `MarkdownPrintTheme`, default `.light`. light, dark, mono,

highContrast.

- `withTOC`: `Bool`, default `false`. Include navigable table of contents.
- `dynamicTypeScale`: `CGFloat`, default `1.0`. Font size multiplier for accessibility.
- `fontFamily`: `FontFamily`, default `.apple`. `.apple` (SF Pro + Menlo) or `.web` (Georgia + Helvetica + Courier).
- `maxMarkdownSize`: `Int`, default `10_000_000`. Max input size in bytes.
- `showLineNumbers`: `Bool`, default `false`. Show line numbers in code blocks.
- `justifyText`: `Bool`, default `false`. Align paragraph text to both margins.
- `watermark`: `Watermark?`, default `nil`. Watermark drawn on every page.
- `headerFooter`: `PageHeaderFooter?`, default `nil`. Page headers and footers.

3.2 `PDFRenderResult`

Returned by `render()`. Contains:

- `pdfData`: `Data`. The generated PDF bytes.
- `pageCount`: `Int`. Number of pages in the final PDF.
- `linkCount`: `Int`. Clickable links detected.
- `imageCount`: `Int`. Images included.
- `headingCount`: `Int`. Headings used for outline and TOC.
- `duration`: `TimeInterval`. Render time in seconds.
- `diagnostics`: `String`. Human-readable summary.

3.3 `MarkdownPageSize`

- `.usLetter`: 612 x 792 pt.
- `.a4`: 595.28 x 841.89 pt.

- `.custom(width:height:)`: any positive point size.

3.4 `PDFMetadata`

```
public struct PDFMetadata {
public let title: String?
public let author: String?
public let subject: String?
public let keywords: [String]
}
```

All fields are optional. `subject` and `keywords` improve document search and identification in PDF managers. The PDF `Creator` field is always set to `"MarkdownPrint"`.

3.5 `MarkdownLayoutElement`

Exposes the layout engine's output for each positioned element:

```
public struct MarkdownLayoutElement {
public let kind: MarkdownLayoutElementKind
public let x, y, width, height: Double
public let fontSize: Double
public let style: MarkdownInlineKind
public let text: String
public let url: String // links and images
public let headingLevel: Int // 0 = body, 1–6 = H1–H6
public let isCodeBlock: Bool
public let isTableCell: Bool
public let isBlockquote: Bool
}
```

Coordinates are relative to the content area (inside margins), with `y` growing downward.

---

4. CLI -- `markdownprint-cli`

```
swift run markdownprint-cli input.md [output.pdf] [options]
```

Options

- `--size a4|letter`: page size. Default: `a4`.
- `--theme light|dark|mono`: visual theme. Default: `light`.
- `--theme-file path.json`: load a custom JSON theme.
- `--high-contrast`: use the high-contrast theme.
- `--toc`: include a table of contents.
- `--title "text"` and `--author "text"`: PDF metadata.
- `--font apple|web`: font preset. Default: `apple`.
- `--justify`: align paragraph text to both margins.
- `--line-numbers`: show line numbers in code blocks.
- `--watermark "TEXT"`: add a text watermark.
- `--watermark-image PATH`: add an image watermark.
- `--header "TEXT"` and `--footer "TEXT"`: page chrome with placeholders.
- `--scale N`: dynamic type scale. Default: `1.0`.

Examples

```
swift run markdownprint-cli document.md
swift run markdownprint-cli input.md output.pdf --size letter --theme dark
swift run markdownprint-cli manual.md --toc --title "Manual v2.0" --author
"Team"
swift run markdownprint-cli blog.md --font web
```

The `--font apple` preset uses San Francisco Pro for headings and body, with Menlo for code blocks. The `--font web` preset uses Georgia for headings, Helvetica for body text, and Courier for code blocks.

---

5. Markdown Support

5.1 Headings (H1-H6)

ATX headings (1-6 `#`):

```
# H1 — 24 pt Bold
## H2 — 18 pt Semibold
### H3 — 15 pt Semibold
#### H4 — 12 pt Semibold
##### H5 — 10.5 pt Semibold
###### H6 — 10.2 pt Semibold (muted)
```

Setext headings (`=` for H1, `-` for H2) are also recognized.

Each heading generates an entry in the PDF outline. H1 and H2 include a subtle underline.

5.2 Inline formatting

Syntax	Result	Font
`**bold**`	**Bold**	SF Pro Bold
`*italic*`	*Italic*	SF Pro Italic
`` `code` ``	`code`	SF Mono + gray background
`~~strikethrough~~`	~~strikethrough~~	SF Pro + line
`[text](url)`	Link	Blue (#0066CC) + clickable
`![alt](url)`	Image	--

HTML inline tags are supported: `<em>`, `<strong>`, `<code>`, `<del>`, `<a hre`, `<img>`, `<br>`.

Automatic small caps : acronyms like `HTML`, `API`, `CSS`, `JSON` inside backticks are automatically rendered with small caps in light and dark themes.

Typography features (all automatic):

- Optical kerning on H1-H3 headings
- Standard ligatures (fi, fl, ff)
- Tabular numbers in table cells (columns align perfectly)

5.3 Paragraphs

Consecutive lines merge into one paragraph. Hard line breaks via two trailing spaces or `<br>`.

5.4 Images

```
![Alt text](path/to/image.png)
```

Supports PNG, JPEG, GIF, WebP, and any CoreGraphics-decodable format. Paths resolve relative to `baseURL`. Data URIs (`...`) are supported. Remote URLs show a placeholder.

Alt text is displayed as a caption below the image.

5.5 Blockquotes

```
> This is a blockquote with *italic* and `code`.
```

Rendered with a left bar (4 pt) and muted text color.

5.6 Lists

Unordered (`-`, `*`, `+`), ordered (`1.`, `2.`), task lists (`- [ ]`, `- [x]`), and n lists (2 spaces per level).

5.7 Tables

```
| Name | Price | Stock |
|-------|-------|-------|
| Alpha | 12.99 | 100 |
| Beta | 8.50 | 45 |
```

Headers in bold with gray background. Full grid. Column alignment via `:---`, `:---:`, `---:`. Large tables split across pages with repeated headers. Tabular numbers for numeric columns.

5.8 Code blocks

```
```swift
func greet(_ name: String) -> String {
return "Hello, \(name)!"
}
```
```

SF Mono 10.2 pt, gray background, rounded corners. Syntax highlighting for Swift, Python, JavaScript/TypeScript, C/C++, Go, Rust, and Shell.

5.9 Math (LaTeX)

Inline: `$E = mc^2$`

Display:

```
$$
\int_{0}^{\infty} e^{-x^2} dx = \frac{\sqrt{\pi}}{2}
$$
```

Equations are rendered natively via the built-in C++ math engine (parser + TeX-style layout engine). No external dependencies required. Supports fractions, radicals, superscripts/subscripts, large operators (sum, product, integral), matrices, accents, Greek letters, and more.

5.10 Other elements

- YAML front-matter (`---` … `---`): silently stripped
- Horizontal rules (`---`, `***`, `___`): full-width line
- Reference links (`[text][ref]` + `[ref]: url`)
- Indented code blocks (4 spaces)

---

6. Themes & Decorations

6.1 Built-in themes

Four built-in themes:

- `.light`: white background, dark text. Default for documents.
- `.dark`: near-black background, light text. Useful in dark-mode apps.
- `.mono`: white background, black text. Print and grayscale friendly.
- `.highContrast`: white background, black text, bold borders. Accessibility focused.

6.2 Custom themes

Create themes by overriding specific properties:

```
let theme = MarkdownPrintTheme.custom(name: "brand", overrides: .init(
text: .hex(0x22, 0x22, 0x44),
linkText: .hex(0xCC, 0x33, 0x00),
codeBackground: .hex(0xF8, 0xF4, 0xEE)
))
```

Load themes from JSON files:

```
let theme = try MarkdownPrintTheme.from(jsonFile: "corporate.json")
```

JSON format:

```
{
"name": "corporate",
"baseTheme": "light",
"colors": {
"text": "#222244",
"linkText": "#CC3300"
},
"underlineLinks": true
}
```

`baseTheme` can be `light`, `dark`, `mono`, or `highContrast`.

6.3 Font families

Family	Headings	Body	Code
`.apple`	SF Pro Display	SF Pro Text	Menlo
`.web`	Georgia	Helvetica	Courier

6.4 Watermarks

Add diagonal text or image watermarks to every page:

```
let result = try engine.render(markdown, options: .init(
watermark: .confidential()
))
// "CONFIDENTIAL" across every page at 45-degree angle

// Custom watermark
let result = try engine.render(markdown, options: .init(
watermark: Watermark(kind: .text("DRAFT"), opacity: 0.1, fontSize: 60)
))

// Image watermark (logo, stamp)
let logo = Watermark.image(at: URL(fileURLWithPath: "logo.png"), opacity:
```

6.5 Headers & Footers

Add page headers and footers with placeholder support:

Placeholder	Replaced by
`{page}`	Current page number
`{total}`	Total page count
`{title}`	Document title from metadata
`{section}`	Current section heading

```
let result = try engine.render(markdown, options: .init(
```

```
headerFooter: .sectionAndPage()
))
// Header: "Section — Page N" on every page

// Custom
let hf = PageHeaderFooter(header: "{title}", footer: "{page}/{total}")
```

6.6 Line numbers & Justification

```
let result = try engine.render(markdown, options: .init(
showLineNumbers: true, // line numbers in code blocks
justifyText: true // align text to both margins
))
```

6.7 Hyphenation

Automatic word hyphenation using the Liang-Knuth algorithm (same as TeX). Supports English and Spanish patterns. Language is auto-detected or set via `RenderOptions`. Long words at line boundaries are split with proper hyphenation points.

6.8 Cross-references & Footnotes

Cross-references link to sections by label:

```
See [Introduction](#introduction) for context.
```

Footnotes are placed at page bottom with a separator line:

```
This claim needs evidence[^1].

[^1]: The supporting citation.
```

---

7. PDF Features

7.1 Navigable outline

All headings appear in the PDF outline panel (Preview, Acrobat). Full

hierarchy: H1 root, H2 child, H3 grandchild.

7.2 Table of contents

Enable with `withTOC: true`. Creates a dedicated TOC page with leader dots and clickable page numbers linking to each heading. The TOC hierarchy mirrors the document outline.

7.3 Clickable links

`[text](url)` and `<a href>` become `PDFActionURL`. Blue (#0066CC), clickable in any PDF viewer.

7.4 Metadata

Field	Source
Title	`PDFMetadata.title`
Author	`PDFMetadata.author`
Creator	`"MarkdownPrint"` (fixed)

7.5 Defenses

- Text clipping within content area
- Long words (URLs, hashes) split before right margin
- Code text clipped within the content area
- Missing images: gray placeholder with alt text, no layout break
- Inline code background: rounded rect with proper padding

---

8. Architecture

Three independent layers:

- Layer 1: `CMarkdownPrintCore`. C++17 parser and layout engine. Cross-platform.
- Layer 2: `MarkdownPrintCore`. Swift wrapper over the C++ core. Cross-platform.
- Layer 3: `MarkdownPrint`. CoreGraphics, CoreText, and PDFKit renderer. Apple only.

Layer 1 -- C++ Core

Lexer, AST builder, inline parser, layout engine, page geometry, approximate font metrics.

Layer 2 -- Swift Bridge

Translates C++ types to Swift via `interoperabilityMode(.Cxx)`. No Apple APIs.

Layer 3 -- Rendering

PDF renderer, system font resolution, syntax highlighting, CLI.

---

9. Standards Compliance

MarkdownPrint implements CommonMark with GFM extensions:

- Tables with column alignment
- Task lists
- Strikethrough
- Reference links
- HTML inline subset (`<em>`, `<strong>`, `<code>`, `<del>`, `<a>`, `<img>`, `<br>`)
- Local images and data URIs
- Fenced code blocks with syntax highlighting

- LaTeX math (inline `$...$` and display `$$...$$`)

Not supported in this version:

- Block HTML (`<div>`, `<table>`, `<pre>`, etc.)
- Automatic remote image download
- MathML equations
- PDF/UA tagged structure (planned)

---

10. Async & Cancellation

Render PDFs asynchronously with Swift Concurrency:

```
let result = try await engine.render(markdown, options: .init(theme: .dark
```

Cancel a long-running render cooperatively:

```
let task = Task {
try await engine.renderPDFCancellable(
fromMarkdown: largeDocument,
pageSize: .a4,
theme: .dark,
withTOC: true,
showLineNumbers: true,
watermark: .draft(),
headerFooter: .sectionAndPage()
)
}
// Later:
task.cancel()
// Throws CancellationError
```

The cancellable API supports all decoration parameters: `showLineNumbers`, `justifyText`, `watermark`, `headerFooter`, and `fontFamily`. Progress reporting also works via the `progress` parameter.

---

11. SwiftUI Integration

`MarkdownPrintView` renders Markdown to PDF in a SwiftUI view:

```
import SwiftUI
import MarkdownPrint

struct ContentView: View {
var body: some View {
MarkdownPrintView("""
# Hello World
This is a **Markdown** preview.
""")
}
}
```

Customize with `MarkdownPrintConfiguration`:

```
let config = MarkdownPrintConfiguration(
pageSize: .usLetter,
theme: .dark,
withTOC: true,
dynamicTypeScale: 1.2,
accessibilityLabel: "Annual report PDF preview"
)
MarkdownPrintView(markdown, configuration: config)
```

`MarkdownPrintView` also reads SwiftUI Dynamic Type from the environment and multiplies it with `dynamicTypeScale`, so larger accessibility text sizes generate larger PDF text. The PDF preview, loading state, and render errors include VoiceOver labels.

Present as a sheet:

```
.markdownPDFPreview(isPresented: $showPDF, markdown: document)
```

`PDFRenderResult` conforms to `Transferable` for drag-and-drop and `ShareLink`.

`MarkdownPrintEngine.shared` provides a singleton for convenience.